

Backend Integration for GenAI RAG Projects

FastAPI · Streaming SSE · Redis Caching

TOPICS COVERED

- FastAPI for LLM Apps — API Layer Design
- REST Endpoints for RAG / LLM Systems
- Streaming (SSE) — Real-time Token Streaming
- Server-Sent Events Architecture
- Caching Layer — Redis for LLM Responses

GenAI Training Series

Table of Contents

1. FastAPI for LLM Apps — *Overview & Architecture*

- 1.1 What is FastAPI?
- 1.2 Why FastAPI for LLM/RAG Applications?
- 1.3 Project Structure for RAG APIs

2. API Layer Design — *REST Endpoints for RAG & LLM Systems*

- 2.1 Core REST Endpoint Patterns
- 2.2 Request / Response Models with Pydantic
- 2.3 Dependency Injection for RAG Components
- 2.4 Error Handling & Status Codes
- 2.5 Authentication & Rate Limiting

3. Streaming with SSE — *Server-Sent Events for Real-Time Responses*

- 3.1 How SSE Works
- 3.2 FastAPI SSE Implementation
- 3.3 Client-Side SSE Consumption
- 3.4 SSE vs WebSockets — When to Use Each

4. Caching Layer — *Redis Caching for LLM Responses*

- 4.1 Why Cache LLM Responses?
- 4.2 Redis Setup & Connection
- 4.3 Semantic Caching Strategy
- 4.4 Cache Invalidation & TTL Policies
- 4.5 Full Integration Example

5. Putting It All Together — *Complete RAG Backend Blueprint*

1. FastAPI for LLM Apps

1.1 What is FastAPI?

FastAPI is a modern, high-performance Python web framework built on top of Starlette (ASGI) and Pydantic. It is designed for building APIs quickly with automatic data validation, serialization, and interactive documentation. FastAPI is particularly well-suited for AI/ML workloads because of its native support for asynchronous programming (async/await), streaming responses, and tight integration with Python's type-hint ecosystem.

Release Year	2018 (Sebastián Ramírez)
Async Support	Native async/await via ASGI (Uvicorn / Hypercorn)
Validation	Pydantic v2 for request & response schema validation
Docs	Auto-generated Swagger UI + ReDoc
Performance	On par with NodeJS and Go in benchmarks
LLM Use-case Fit	Streaming, async calls to OpenAI/Anthropic/local LLMs, SSE

1.2 Why FastAPI for LLM/RAG Applications?

Large Language Model APIs are inherently slow (several seconds per response). FastAPI's non-blocking I/O model allows your server to handle hundreds of concurrent LLM calls without spawning expensive threads. The framework's streaming response support is critical for delivering token-by-token output to end users.

Advantages for GenAI	Advantages for RAG
Async I/O: Non-blocking calls to LLM APIs (OpenAI, Bedrock, etc.)	Auto Docs: Swagger UI allows rapid testing during development
Streaming: StreamingResponse + SSE built into the framework	Middleware: Easy auth, CORS, rate-limiting via middleware stack
Pydantic: Strict schema enforcement on complex RAG request bodies	Background Tasks: Trigger indexing / caching without blocking
DI System: Inject vector stores, LLM clients, embedders cleanly	Ecosystem: Works natively with LangChain, LlamaIndex, OpenAI SDK

1.3 Project Structure for RAG APIs

A well-organised project structure keeps RAG API concerns separated. Below is the recommended directory layout for a production-grade GenAI RAG backend:

Recommended RAG Backend Directory Layout

rag_backend/

```

■■■ main.py                # FastAPI app entry-point
■■■ config.py              # Settings (env vars, model names)
■■■ models/
■   ■■■ request_models.py  # Pydantic input schemas
■   ■■■ response_models.py # Pydantic output schemas
■■■ routers/
■   ■■■ query.py           # /query, /stream endpoints
■   ■■■ documents.py       # /ingest, /delete endpoints
■   ■■■ health.py         # /health, /metrics
■■■ services/
■   ■■■ llm_service.py     # LLM client wrapper
■   ■■■ retrieval_service.py # Vector store queries
■   ■■■ cache_service.py   # Redis caching logic
■   ■■■ embedding_service.py # Embedding generation
■■■ middleware/
■   ■■■ auth.py            # API key / JWT validation
■   ■■■ rate_limit.py      # Request rate limiting
■■■ utils/
    ■■■ prompt_builder.py  # Prompt template assembly

```

main.py – Application Bootstrap

```

# main.py – Application bootstrap

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from routers import query, documents, health
from middleware.auth import APIKeyMiddleware

app = FastAPI(
    title="GenAI RAG Backend",
    description="REST + Streaming API for RAG-powered LLM applications",
    version="1.0.0",
)

# Middleware stack
app.add_middleware(CORSMiddleware, allow_origins=['*'],
                  allow_methods=['*'], allow_headers=['*'])
app.add_middleware(APIKeyMiddleware)

# Router registration
app.include_router(query.router, prefix='/api/v1', tags=['Query'])
app.include_router(documents.router, prefix='/api/v1', tags=['Documents'])

```

```
app.include_router(health.router, prefix='/api/v1', tags=['Health'])
```

2. API Layer Design

The API layer is the contract between your RAG backend and its consumers (web apps, mobile clients, other microservices). Good API design makes a RAG system maintainable, scalable, and easy to integrate. This section covers endpoint design, request/response models, dependency injection, error handling, and security.

2.1 Core REST Endpoint Patterns

Endpoint	Method	Description
/api/v1/query	POST	Single-turn RAG query — returns full answer
/api/v1/stream	POST	SSE streaming query — token-by-token response
/api/v1/chat	POST	Multi-turn chat with history context
/api/v1/ingest	POST	Upload & index documents into vector store
/api/v1/documents/{id}	DELETE	Remove a document from the index
/api/v1/documents	GET	List all indexed documents with metadata
/api/v1/health	GET	Liveness check — returns 200 if healthy
/api/v1/metrics	GET	Prometheus-style operational metrics

2.2 Request / Response Models with Pydantic

Pydantic models enforce strict typing at the API boundary. They automatically parse incoming JSON, validate field types/constraints, and generate the OpenAPI schema for Swagger docs. All RAG request and response bodies should be defined as Pydantic BaseModel subclasses.

Request Models

```
# models/request_models.py

from pydantic import BaseModel, Field
from typing import Optional, List

class QueryRequest(BaseModel):
    question: str = Field(..., min_length=3,
                           description="User question for RAG")
    top_k: int = Field(default=5, ge=1, le=20,
                       description="Number of chunks to retrieve")
    temperature: float = Field(default=0.3, ge=0.0, le=2.0)
    filters: Optional[dict] = Field(default=None,
                                     description="Metadata filters for retrieval")

class ChatRequest(BaseModel):
```

```

messages: List[dict] = Field(...,
                               description="[{role, content}, ...]")

top_k: int = Field(default=5)

session_id: Optional[str] = Field(default=None)

class IngestRequest(BaseModel):
    text: str = Field(..., description="Document text to index")
    metadata: Optional[dict] = Field(default={},
                                     description="doc_id, title, source, etc.")

```

Response Models

```

# models/response_models.py

from pydantic import BaseModel
from typing import List, Optional

class SourceChunk(BaseModel):
    chunk_id: str
    text: str
    score: float
    metadata: dict

class QueryResponse(BaseModel):
    answer: str
    sources: List[SourceChunk]
    model_used: str
    latency_ms: int
    cached: bool = False

class IngestResponse(BaseModel):
    doc_id: str
    chunks_indexed: int
    status: str # 'success' | 'partial' | 'failed'

```

2.3 Dependency Injection for RAG Components

FastAPI's Depends() system allows you to declare shared resources — vector store clients, LLM instances, Redis connections — once and inject them into any route handler. This pattern avoids global state and makes unit testing straightforward.

Dependency Injection Pattern

```

# services/dependencies.py

from functools import lru_cache
from fastapi import Depends

```

```

from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
from redis.asyncio import Redis
from config import Settings

@lru_cache()
def get_settings() -> Settings:
    return Settings()

async def get_llm(settings: Settings = Depends(get_settings)):
    return ChatOpenAI(
        model=settings.LLM_MODEL,
        api_key=settings.OPENAI_API_KEY,
        streaming=True
    )

async def get_vector_store(settings: Settings = Depends(get_settings)):
    embedder = OpenAIEmbeddings(api_key=settings.OPENAI_API_KEY)
    return FAISS.load_local(settings.FAISS_INDEX_PATH, embedder)

async def get_redis(settings: Settings = Depends(get_settings)):
    return Redis.from_url(settings.REDIS_URL, decode_responses=True)

# Usage in a router:
from fastapi import APIRouter
router = APIRouter()

@router.post('/query')
async def query_rag(
    req: QueryRequest,
    llm = Depends(get_llm),
    vstore = Depends(get_vector_store),
    redis = Depends(get_redis)
):
    ... # all dependencies auto-resolved

```

2.4 Error Handling & HTTP Status Codes

HTTP Code	Scenario	FastAPI Handling
400 Bad Request	Invalid query / missing field	Pydantic raises RequestValidationError automatically
401 Unauthorized	Missing or invalid API key	Raise HTTPException(401) in auth middleware
404 Not Found	Document ID doesn't exist	Raise HTTPException(404, detail='doc not found')

422 Unprocessable Entity	Schema validation failure	FastAPI returns this automatically with field details
429 Too Many Requests	Rate limit exceeded	Raise HTTPException(429) in rate-limit middleware
503 Service Unavailable	LLM API down / timeout	Catch httpx.TimeoutException → 503

Exception Handlers

```
# Centralised exception handler

from fastapi import Request
from fastapi.responses import JSONResponse
from fastapi.exceptions import RequestValidationError

@app.exception_handler(RequestValidationError)
async def validation_exception_handler(request: Request, exc):
    return JSONResponse(
        status_code=422,
        content={
            "error": "Validation Error",
            "detail": exc.errors(),
            "body": exc.body
        }
    )

@app.exception_handler(Exception)
async def general_exception_handler(request: Request, exc: Exception):
    return JSONResponse(
        status_code=500,
        content={"error": "Internal Server Error", "message": str(exc)}
    )
```

3. Streaming with Server-Sent Events (SSE)

Large Language Models generate text token-by-token. Without streaming, the user stares at a blank screen for 10-30 seconds before seeing any output. Server-Sent Events (SSE) allow the FastAPI server to push each token to the browser as it is generated, creating the familiar 'typing' effect seen in ChatGPT and Claude.

3.1 How SSE Works

KEY CONCEPT: SSE is a unidirectional, text-based protocol over a standard HTTP connection. The server holds the connection open and pushes data: lines. The browser's EventSource API listens for these events without polling.

Step	Action	Detail
1	Client sends POST /stream	Includes question JSON in request body
2	Server opens SSE connection	Content-Type: text/event-stream
3	Server calls LLM (streaming)	Uses stream=True with OpenAI / LangChain
4	Each token received	Server writes: data: {token}\n\n
5	LLM finishes	Server writes: data: [DONE]\n\n
6	Client closes stream	EventSource.close() called on [DONE]

3.2 FastAPI SSE Implementation

FastAPI supports SSE natively through its StreamingResponse class combined with async generators. The key is to use the LLM's streaming callback or async iteration to yield chunks as they arrive from the model.

FastAPI SSE Streaming Endpoint

```
# routers/query.py – SSE Streaming Endpoint

import json

from fastapi import APIRouter, Depends
from fastapi.responses import StreamingResponse
from models.request_models import QueryRequest
from services.dependencies import get_llm, get_vector_store
from utils.prompt_builder import build_rag_prompt

router = APIRouter()

async def sse_generator(question: str, top_k: int, llm, vstore):
    """Async generator that yields SSE-formatted token strings."""
    try:
        # Step 1: Retrieve relevant chunks
```

```

docs = await vstore.asimilarity_search(question, k=top_k)
context = '\n\n'.join([d.page_content for d in docs])

# Step 2: Build augmented prompt
prompt = build_rag_prompt(question, context)

# Step 3: Stream from LLM, yielding each token
async for chunk in llm.astream(prompt):
    token = chunk.content
    if token:
        payload = json.dumps({'token': token, 'done': False})
        yield f"data: {payload}\n\n"

# Step 4: Send completion signal with sources
sources = [{'text': d.page_content[:150],
            'metadata': d.metadata} for d in docs]
done_payload = json.dumps({'token': '', 'done': True,
                           'sources': sources})

yield f"data: {done_payload}\n\n"

except Exception as e:
    error_payload = json.dumps({'error': str(e), 'done': True})
    yield f"data: {error_payload}\n\n"

@router.post('/stream')
async def stream_query(
    req: QueryRequest,
    llm = Depends(get_llm),
    vstore = Depends(get_vector_store)
):
    return StreamingResponse(
        sse_generator(req.question, req.top_k, llm, vstore),
        media_type="text/event-stream",
        headers={
            "Cache-Control": "no-cache",
            "X-Accel-Buffering": "no", # Disable Nginx buffering
            "Connection": "keep-alive"
        }
    )

```

3.3 Client-Side SSE Consumption

The browser's native EventSource API cannot send POST requests with a body. For RAG queries (which require a question in the request body), use the `fetch()` API with the `ReadableStream` interface or the `@microsoft/fetch-event-source` library.

JavaScript – Consuming SSE with `fetch()`

```
// JavaScript – Consuming SSE via fetch() + ReadableStream
async function streamQuery(question) {
  const response = await fetch('/api/v1/stream', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json',
              'X-API-Key': API_KEY },
    body: JSON.stringify({ question, top_k: 5 })
  });

  const reader = response.body.getReader();
  const decoder = new TextDecoder();
  let buffer = '';

  while (true) {
    const { done, value } = await reader.read();
    if (done) break;

    buffer += decoder.decode(value, { stream: true });
    const lines = buffer.split('\n');
    buffer = lines.pop(); // Keep incomplete line

    for (const line of lines) {
      if (line.startsWith('data: ')) {
        const data = JSON.parse(line.slice(6));
        if (data.done) {
          renderSources(data.sources);
          reader.cancel();
        } else {
          appendToken(data.token); // Update UI incrementally
        }
      }
    }
  }
}
```

Python Client – Async SSE Consumer

```
# Python client – Consuming SSE with httpx
```

```

import httpx, json

async def stream_rag_query(question: str):
    async with httpx.AsyncClient() as client:
        async with client.stream(
            'POST', 'http://localhost:8000/api/v1/stream',
            json={'question': question, 'top_k': 5},
            headers={'X-API-Key': API_KEY},
            timeout=120
        ) as response:
            async for line in response.aiter_lines():
                if line.startswith('data: '):
                    data = json.loads(line[6:])
                    if data.get('done'):
                        print('\n--- Sources ---')
                        for src in data.get('sources', []):
                            print(src['text'][:80])
                        break
                    else:
                        print(data['token'], end='', flush=True)

```

3.4 SSE vs WebSockets — When to Use Each

Criterion	SSE (Server-Sent Events)	WebSockets
Direction	Server → Client only	Bidirectional (full duplex)
Protocol	HTTP/1.1 or HTTP/2	WS / WSS upgrade
Browser Support	All modern browsers native	All modern browsers
Reconnection	Automatic (built-in)	Manual implementation required
LLM Streaming Fit	Ideal — push tokens to client	Possible but overkill for tokens
Multi-turn Chat	New SSE per turn	Single socket, send/receive msgs
Load Balancer	Works with sticky sessions	Needs WS-aware load balancer
Recommended For	Token streaming, notifications	Real-time chat, collaborative tools

RECOMMENDATION: For GenAI RAG applications, SSE is the recommended choice for streaming LLM token output. Use WebSockets only if you need real-time bidirectional features like collaborative document editing or multi-user chat alongside the LLM.

4. Caching Layer — Redis for LLM Responses

LLM API calls are expensive in both cost and latency. A well-designed caching layer can reduce costs by 40-70% on production RAG workloads while improving P95 response times from 8-15s to under 50ms for repeated queries. Redis is the industry standard for LLM response caching due to its sub-millisecond read latency and rich data structures.

4.1 Why Cache LLM Responses?

Metric	Without Cache	With Redis Cache
Repeated Query Latency	8 – 15 seconds	< 50 milliseconds
LLM API Cost (1M queries)	~\$2,000 – \$8,000	~\$600 – \$2,400 (70% savings)
LLM API Rate Limit Risk	High on bursty traffic	Minimal (cache absorbs spikes)
User Experience	Spinner on every request	Instant on cache hits
Token Usage	Full tokens each request	Zero tokens on cache hit

4.2 Redis Setup & Connection

Use the redis-py library with async support (redis.asyncio) to integrate Redis into FastAPI without blocking the event loop. Connection pooling is handled automatically.

Redis Setup & Connection Pooling

```
# Install dependencies
# pip install redis[hiredis] aioredis

# config.py – Settings
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    REDIS_URL: str = "redis://localhost:6379/0"
    CACHE_TTL_SECONDS: int = 3600 # 1 hour default
    CACHE_MAX_SIZE_MB: int = 512
    SEMANTIC_CACHE_THRESHOLD: float = 0.92 # cosine similarity

    class Config:
        env_file = ".env"

# services/cache_service.py – Redis Client
from redis.asyncio import Redis, ConnectionPool
from config import Settings

settings = Settings()
```

```

pool = ConnectionPool.from_url(
    settings.REDIS_URL,
    max_connections=20,
    decode_responses=True
)

async def get_redis_client() -> Redis:
    return Redis(connection_pool=pool)

# Health-check helper
async def ping_redis() -> bool:
    try:
        r = await get_redis_client()
        return await r.ping()
    except Exception:
        return False

```

4.3 Exact Cache Strategy (Deterministic Lookup)

The simplest caching strategy hashes the exact query string and model parameters to produce a cache key. This works well for FAQ bots and support agents where users tend to ask identical questions repeatedly.

Exact Cache – Deterministic Key Strategy

```

import hashlib, json
from redis.asyncio import Redis
from config import Settings

settings = Settings()

def make_cache_key(question: str, top_k: int, filters: dict) -> str:
    """Deterministic hash of all query parameters."""
    payload = json.dumps({
        "q": question.strip().lower(),
        "k": top_k,
        "f": filters or {}
    }, sort_keys=True)
    return "rag:v1:" + hashlib.sha256(payload.encode()).hexdigest()[:32]

class ExactCacheService:
    def __init__(self, redis: Redis):
        self.redis = redis
        self.ttl = settings.CACHE_TTL_SECONDS

    async def get(self, key: str) -> dict | None:

```

```

        raw = await self.redis.get(key)
        return json.loads(raw) if raw else None

    async def set(self, key: str, value: dict) -> None:
        await self.redis.setex(key, self.ttl, json.dumps(value))

    async def invalidate(self, pattern: str) -> int:
        """Delete all keys matching pattern, e.g. rag:v1:*****"""
        keys = await self.redis.keys(pattern)
        if keys:
            return await self.redis.delete(*keys)
        return 0

```

4.4 Semantic Caching Strategy

Exact caching misses paraphrase variations: 'What is RAG?' and 'Explain RAG to me' are semantically identical but produce different hash keys. Semantic caching stores query embeddings in Redis and performs cosine similarity search before calling the LLM.

HOW IT WORKS: Semantic cache works in two phases: (1) Embed the incoming question, (2) Search Redis for a stored embedding with cosine similarity above threshold (e.g. 0.92). If found, return the cached answer. Otherwise, call the LLM and store the new embedding + answer pair.

Semantic Cache with Embedding Similarity

```

import numpy as np
from openai import AsyncOpenAI
from redis.asyncio import Redis
import json

class SemanticCacheService:
    EMBED_MODEL = "text-embedding-3-small"
    KEY_PREFIX = "sem_cache:"

    def __init__(self, redis: Redis, threshold: float = 0.92):
        self.redis = redis
        self.threshold = threshold
        self.oai = AsyncOpenAI()

    async def _embed(self, text: str) -> list[float]:
        resp = await self.oai.embeddings.create(
            model=self.EMBED_MODEL, input=text
        )
        return resp.data[0].embedding

    @staticmethod

```



```

def _cosine(a: list, b: list) -> float:
    a, b = np.array(a), np.array(b)
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

async def lookup(self, question: str) -> dict | None:
    q_emb = await self._embed(question)
    keys = await self.redis.keys(self.KEY_PREFIX + "*")
    best_score, best_key = 0.0, None
    for key in keys:
        raw = await self.redis.hget(key, 'embedding')
        if not raw: continue
        stored_emb = json.loads(raw)
        score = self._cosine(q_emb, stored_emb)
        if score > best_score:
            best_score, best_key = score, key
    if best_score >= self.threshold and best_key:
        answer_raw = await self.redis.hget(best_key, 'answer')
        return json.loads(answer_raw) if answer_raw else None
    return None

async def store(self, question: str, answer: dict) -> None:
    q_emb = await self._embed(question)
    import uuid
    key = self.KEY_PREFIX + str(uuid.uuid4())[0:16]
    await self.redis.hset(key, mapping={
        'embedding': json.dumps(q_emb),
        'answer': json.dumps(answer),
        'question': question
    })
    await self.redis.expire(key, 7200)

```

4.5 Cache Invalidation & TTL Policies

Cache Type	Recommended TTL	Invalidation Trigger
FAQ / Static answers	24 – 72 hours	Manual flush on content update
Document-grounded RAG	1 – 4 hours	On document ingest/delete
Real-time data (news, prices)	5 – 15 minutes	Time-based expiry only
User-specific context	30 minutes (session)	On user session end
Semantic cache embeddings	2 – 6 hours	On model version change

This example combines the exact cache check, vector retrieval, LLM call, cache store, and structured response into a single production-ready endpoint.

```
# routers/query.py – Full non-streaming endpoint

import time

from fastapi import APIRouter, Depends, HTTPException

from models.request_models import QueryRequest

from models.response_models import QueryResponse, SourceChunk

from services.dependencies import get_llm, get_vector_store, get_redis

from services.cache_service import ExactCacheService, make_cache_key

from utils.prompt_builder import build_rag_prompt

router = APIRouter()

@router.post('/query', response_model=QueryResponse)

async def query_rag(

    req: QueryRequest,

    llm = Depends(get_llm),

    vstore = Depends(get_vector_store),

    redis = Depends(get_redis),

):

    start = time.monotonic()

    cache = ExactCacheService(redis)

    ckey = make_cache_key(req.question, req.top_k, req.filters)

    # █ Cache lookup █

    cached = await cache.get(ckey)

    if cached:

        cached['latency_ms'] = int((time.monotonic() - start) * 1000)

        cached['cached'] = True

        return QueryResponse(**cached)

    # █ Retrieval █

    docs = await vstore.asimilarity_search_with_score(

        req.question, k=req.top_k,

        filter=req.filters
```

Page 19

Layer	Technology	Responsibility
HTTP / Transport	FastAPI + Uvicorn	Receive requests, CORS, middleware chain
Authentication	API Key / JWT Middleware	Validate X-API-Key or Bearer token
Rate Limiting	SlowAPI (token bucket)	Protect LLM budget per user/IP
Request Parsing	Pydantic BaseModel	Validate & type-check all inputs
Cache (L1 Exact)	Redis + SHA-256 key	Instant hit on identical queries
Cache (L2 Semantic)	Redis + Embeddings	Near-duplicate query matching
Retrieval	FAISS / Pinecone / Weaviate	Top-K chunk retrieval by similarity
Prompt Assembly	LangChain / Custom	Inject context + format system prompt
LLM Inference	OpenAI / Azure / Bedrock	Generate answer tokens

Streaming	StreamingResponse + SSE	Push tokens to client in real-time
Response Model	Pydantic QueryResponse	Structure & validate output
Observability	Prometheus + Grafana	Track latency, cache hit rate, tokens

5.3 Quick-Start: Running Locally

Local Development Quick-Start

```
# 1. Clone and install dependencies
git clone https://github.com/your-org/rag-backend
cd rag-backend
pip install fastapi uvicorn[standard] langchain langchain-openai \
    faiss-cpu redis[hiredis] pydantic-settings openai

# 2. Start Redis (Docker)
docker run -d -p 6379:6379 --name redis redis:7-alpine

# 3. Set environment variables
export OPENAI_API_KEY=sk-...
export REDIS_URL=redis://localhost:6379/0
export FAISS_INDEX_PATH=./data/faiss_index

# 4. Launch the API server
uvicorn main:app --host 0.0.0.0 --port 8000 --reload

# 5. Test the health endpoint
curl http://localhost:8000/api/v1/health

# 6. Test a RAG query
curl -X POST http://localhost:8000/api/v1/query \
    -H 'Content-Type: application/json' \
    -H 'X-API-Key: test-key' \
    -d '{"question": "What is retrieval augmented generation?", "top_k": 5}'

# 7. Open Swagger docs
open http://localhost:8000/docs
```

SUMMARY: The complete backend blueprint covers FastAPI API design, Pydantic validation, dependency injection, SSE streaming, exact + semantic Redis caching, error handling, and authentication. Each component is independently testable and can be swapped for alternative implementations (e.g. Pinecone instead of FAISS, Anthropic instead of OpenAI).